

Task lec25

1. Laravel Framework

What is Laravel?

Laravel is a free, open-source PHP web application framework designed to simplify web development by providing elegant syntax and powerful built-in features. It was created by Taylor Otwell and follows the Model-View-Controller (MVC) architectural pattern.

Key Features of Laravel

- MVC Architecture
- Routing System
- Eloquent ORM
- Blade Templating Engine
- Authentication and Authorization
- Middleware Support
- Database Migration
- Dependency Injection
- Service Container
- Security Features

Advantages of Laravel

- Clean and readable code.
 - Faster development process.
 - Strong security mechanisms.
 - Large community support.
 - Extensive documentation.
-

2. MVC Architecture

What is MVC?

MVC stands for Model-View-Controller. It is a software architectural pattern used to separate application logic from the user interface.

Components of MVC

Model

The Model is responsible for managing data and business logic. It interacts with the database and performs operations such as creating, reading, updating, and deleting records.

Example:

```
class User extends Model
{
    protected $fillable = ['name', 'email'];
}
```

View

The View is responsible for displaying data to users. Laravel uses the Blade templating engine to create views.

Example:

```
<h1>Welcome, {{ $user->name }}</h1>
```

Controller

The Controller acts as an intermediary between the Model and the View. It processes user requests and returns responses.

Example:

```
class UserController extends Controller
{
    public function index()
    {
        $users = User::all();
        return view('users.index', compact('users'));
    }
}
```

Benefits of MVC

- Better code organization.
- Easier maintenance.
- Improved scalability.
- Separation of concerns.

3. Laravel Request Lifecycle

What is the Request Lifecycle?

The Request Lifecycle describes the path a user request follows from the moment it enters the application until a response is returned.

Steps of Laravel Request Lifecycle

Step 1: Entry Point

All requests enter through the `public/index.php` file.

Step 2: HTTP Kernel

The request is passed to Laravel's HTTP Kernel, which handles middleware execution.

Step 3: Service Providers

Laravel loads and registers all configured service providers.

Step 4: Routing

The router determines which route matches the incoming request.

Step 5: Middleware Execution

Middleware checks and filters the request before reaching the controller.

Step 6: Controller Action

The appropriate controller method is executed.

Step 7: Response Generation

The controller returns a response, view, or JSON data.

Step 8: Send Response

Laravel sends the final response back to the client.

Request Lifecycle Flow

User Request

↓

`public/index.php`

↓

HTTP Kernel

↓

Service Providers

↓

Router

↓

Middleware

↓

Controller

↓

Response

↓

Browser

4. Service Providers in Laravel

What are Service Providers?

Service Providers are the central place where Laravel bootstraps and configures application services. Almost every Laravel component is registered through a service provider.

Purpose of Service Providers

- Register services into the service container.
- Bind interfaces to implementations.
- Configure application services.
- Register event listeners and middleware.

Main Methods

register()

Used to bind services into the service container.

Example:

```
public function register()
{
    $this->app->bind(
        PaymentInterface::class,
        PaymentService::class
    );
}
```

boot()

Executed after all services are registered.

Example:

```
public function boot()
{
    View::share('appName', 'My Laravel App');
}
```

Types of Service Providers

- Core Service Providers
- Custom Service Providers

- Package Service Providers

Benefits

- Centralized configuration.
 - Better dependency management.
 - Improved maintainability.
-

5. Middleware in Laravel

What is Middleware?

Middleware acts as a filter between an HTTP request and the application. It allows developers to inspect, modify, or reject requests before they reach controllers.

Common Uses

- Authentication
- Authorization
- Logging
- CORS Handling
- Request Validation
- Rate Limiting

Middleware Example

```
public function handle($request, Closure $next)
{
    if (!auth()->check()) {
        return redirect('/login');
    }

    return $next($request);
}
```

How Middleware Works

```
Request
↓
Middleware
↓
Controller
↓
```

Response



Middleware



Browser

Types of Middleware

Global Middleware

Runs on every request.

Route Middleware

Applied to specific routes.

Example:

```
Route::get('/dashboard', function () {  
    return view('dashboard');  
})->middleware('auth');
```

Advantages of Middleware

- Improves security.
 - Keeps controllers clean.
 - Promotes code reusability.
 - Centralizes request filtering.
-